DESENVOLVIMENTO MOBILE · MÓDULO 06 · TRILHA FLUTTER · NÍVEL INICIANTE

Dart Assíncrono

Event loop, Future e async/await, tratamento de erros, Stream, StreamController, FutureBuilder/StreamBuilder e Isolates

Prof. Dr. Francisco dos Santos Viana

São Luís – Maranhão – Brasil · 2026

Objetivos do módulo

Ao final deste módulo você será capaz de

- Explicar o *event loop* do Dart e por que a interface não pode ser bloqueada.
- Escrever código assíncrono com `Future`, `async` e `await`, tratando erros e tempos limite.
- Executar tarefas em paralelo com `Future.wait`.
- Produzir e consumir fluxos de dados com `Stream`, `async*` e `StreamController`.
- Mostrar dados assíncronos na tela com `FutureBuilder` e `StreamBuilder`.
- Levar cálculos pesados para outro *isolate*.

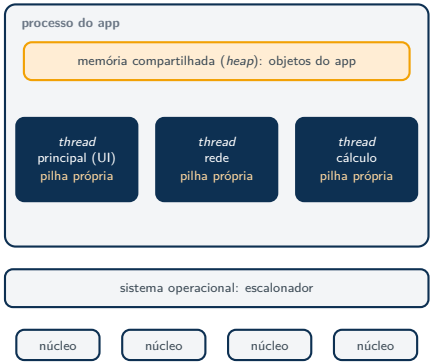
Roteiro

Neste módulo

- Fundamentos: concorrência e assincronia
- O event loop
- Future e async/await
- Streams
- Assíncrono na interface
- Isolates
- Fechamento

Fundamentos: concorrência e assincronia

Programas, processos e *threads*



Processo e *thread*

Processo: programa em execução, com memória própria e isolada dos outros apps. ***Thread***: linha de execução dentro do processo; as *threads* dividem a memória do processo, mas cada uma tem sua pilha.

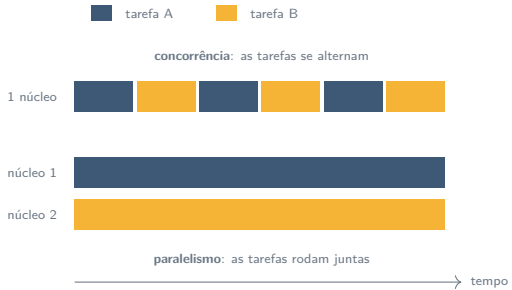
O escalonador

O sistema operacional distribui as *threads* pelos núcleos e alterna entre elas milhares de vezes por segundo (troca de contexto). Cada *thread* custa memória e tempo de troca: não se cria uma por tarefa.

Analogia

O processo é a cozinha; as *threads* são os cozinheiros. Eles dividem a despensa, e é aí que começam as brigas pelo mesmo ingrediente.

Concorrência e paralelismo



Concorrência

Várias tarefas em andamento no mesmo período, que podem se alternar. É um jeito de **estruturar** o programa.

Paralelismo

Várias tarefas executando **no mesmo instante**, em núcleos diferentes. É um jeito de **executar**.

Chips de celular

Combinam núcleos de alto desempenho e núcleos econômicos (big.LITTLE): o sistema escolhe onde cada *thread* roda para poupar bateria.

Na frase de Rob Pike (2012)

Concorrência é lidar com muitas coisas ao mesmo tempo; paralelismo é fazer muitas coisas ao mesmo tempo.

Tarefas de I/O e tarefas de CPU

Tipo	O que limita	Exemplos	Kotlin	Dart
I/O	espera por rede, disco ou banco	requisição HTTP, consulta SQL, ler arquivo	`suspend` + `Dispatchers.IO`	`Future` + `await`
CPU	cálculo no processador	decodificar imagem, JSON gigante, criptografia	`Dispatchers.Default`	`Isolate.run`
Tempo	o relógio	atraso, repetição, tempo limite	`delay`, `withTimeout`	`Future.delayed`, `Timer`

Bloqueante *versus* não bloqueante

Uma chamada **bloqueante** prende a *thread* até terminar. Uma **não bloqueante** registra o pedido e libera a *thread*; o resultado chega depois, por notificação do sistema.

A ideia-chave do módulo

Esperar não precisa de *thread*: enquanto a resposta não chega, a mesma *thread* atende outros eventos. Já o **cálculo pesado precisa de um núcleo** de verdade, fora da *thread* da interface.

O laço de eventos em cada plataforma

Laço de eventos (*event loop*)

Uma *thread* que repete sem parar: pega o próximo evento da fila, executa o código associado até o fim e volta à fila. Toda interface gráfica funciona assim.

Plataforma	Mecanismo na *thread* principal
Android	`Looper` + `MessageQueue` + `Handler`
iOS	`RunLoop` e fila principal do GCD
Dart	fila de microtarefas e fila de eventos
Web	laço de eventos do navegador (tarefas e microtarefas)

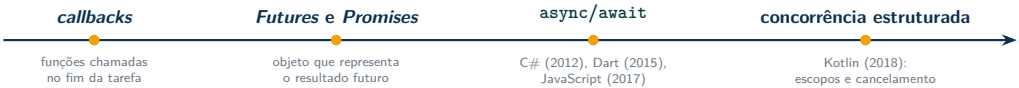
Dart: duas filas

Antes de cada evento, o Dart esvazia a fila de **microtarefas** (continuações curtas de `Futures`). Desde o Flutter 3.29, no Android e no iOS, o código Dart roda na própria *thread* principal do app.

Um evento lento atrasa todos

Quadros se perdem (*jank*) e, no Android, sem resposta a um toque por cerca de 5 s aparece o diálogo ANR.

Da pirâmide de *callbacks* ao `async/await`



Future (ou *Promise*)

Objeto que representa um resultado que ainda não existe. Começa **pendente** e termina **com valor** ou **com erro**. Quem o recebe pode esperar sem bloquear a *thread*.



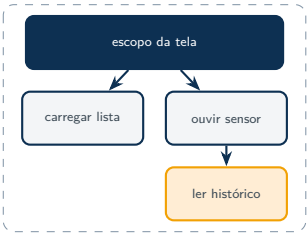
O que acontece em um `await`

A função pausa em um **ponto de suspensão** e devolve a *thread*; quando o resultado chega, continua de onde parou, com as variáveis locais preservadas. O compilador do Kotlin transforma cada função `suspend` em uma máquina de estados que recebe uma `Continuation`.

Dica

O ganho é escrever código assíncrono com cara de sequencial: `try`/`catch` e laços funcionam como sempre.

Concorrência estruturada



a tela fecha ⇒ o escopo termina
⇒ as tarefas filhas são canceladas

Um pouco de história

Assim como a programação estruturada aposentou o `goto`, este modelo aposenta tarefas soltas. A ideia ganhou força em 2018, com um ensaio de Nathaniel J. Smith, e o Kotlin a adotou no mesmo ano.

Concorrência estruturada

Toda tarefa concorrente nasce dentro de um **escopo** e não vive mais que ele: quando o escopo termina, as tarefas filhas são canceladas.

Garantias

Nada fica rodando depois que a tela fecha; o cancelamento chega aos filhos; erros dos filhos chegam ao pai.

E no Dart?

`Future` e `Stream` não seguem esse modelo: cancele inscrições e temporizadores no `dispose` e confira `mounted` após um `await`.

Condições de corrida e impasses



duas somas de 1 a partir de 5 deveriam dar 7: uma atualização se perdeu

Condição de corrida

Erro em que o resultado depende da ordem imprevisível em que tarefas concorrentes acessam um dado compartilhado.

Impasse (*deadlock*)

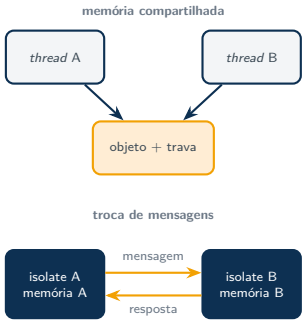
Duas tarefas, cada uma segurando um recurso e esperando o da outra: ninguém avança. Adquira travas sempre na mesma ordem.

Proteção	Como funciona
Exclusão mútua	uma tarefa por vez na região crítica (`Mutex`)
Operação atômica	ler e gravar em um passo indivisível (`AtomicInteger`, `StateFlow.update`)
Confinamento	só uma *thread* toca o dado (a principal, por exemplo)
Imutabilidade	ninguém altera: cria-se uma cópia nova
Mensagens	cada tarefa tem seus dados e troca cópias

Corrida lógica com uma *thread* só

Entre dois `await`, outro evento pode mudar o estado. Duas buscas disparadas em sequência podem responder fora de ordem, e a mais antiga sobrescrever a mais nova.

Memória compartilhada ou troca de mensagens



	Memória compartilhada	Troca de mensagens
Exemplos	*threads* da JVM (Kotlin, Java)	*isolates* do Dart, atores, Web Workers
Comunicação	ler e gravar o mesmo objeto	enviar cópias de dados
Risco	corridas e impasses	custo de copiar dados grandes
Proteção	travas, atômicos, confinamento	o próprio modelo isola

Analogia

Um quadro branco compartilhado exige regras para ninguém apagar o do colega. Colegas que trocam bilhetes nunca brigam pelo quadro, mas gastam tempo escrevendo.

Fluxos de dados: frios, quentes e contrapressão

Fluxo (*stream*)

Sequência de valores que chegam ao longo do tempo: leituras de sensor, mensagens, mudanças no banco. Segue o padrão **Observador**: quem produz notifica quem se inscreveu.

Tipo	Comportamento	Kotlin	Dart
Frio	o produtor começa a cada inscrição	`Flow`	`Stream` (`async*`)
Quente	existe sem observadores; quem chega tarde perde o passado	`SharedFlow`	`Stream` *broadcast*
Estado	quente e guarda o valor atual	`StateFlow`	`ValueNotifier`

Contrapressão (*backpressure*)

O que fazer quando o produtor é mais rápido que o consumidor: acumular (*buffer*), descartar, manter só o mais recente (*conflate*) ou amostrar (*debounce*, *sample*).

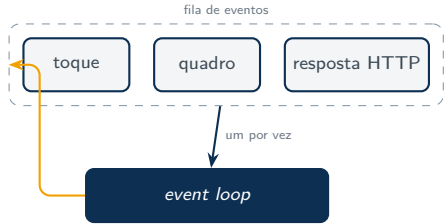
No LabMonitor

Um sensor envia 100 leituras por segundo e a tela desenha 60 quadros: basta mostrar sempre a leitura mais recente.

O event loop

Uma única thread, sem travar

- O código Dart do app roda em um **isolate** principal, com uma única thread.
- O **event loop** processa, um de cada vez, toques, quadros a desenhar, respostas de rede e temporizadores.
- Para 60 Hz, cada quadro tem cerca de **16 ms**. Uma tarefa longa atrasa os quadros e a tela "engasga" (*jank*).
- Operações de I/O (rede, arquivo, banco) são **assíncronas**: o Dart pede ao sistema e segue atendendo eventos até a resposta chegar.



Atenção

I/O não trava a UI se for assíncrono. Já um laço de CPU pesado trava, mesmo dentro de uma função `async`: para isso existem os *isolates*.

Future e async/await

Future, async e await

future.dart

```dart
Future<double> lerTemperatura() async {
  await Future.delayed(const Duration(milliseconds: 500));
  return 24.7;
}

Future<String> relatorio() async {
  final t = await lerTemperatura();   // espera sem travar
  return 'Temperatura: $t °C';
}

void main() async {
  print('antes');
  final r = await relatorio();
  print(r);
  print('depois');
}
```

Future<T>

Promessa de um valor do tipo `T` (ou de um erro) que chegará no futuro. Uma função `async` sempre devolve um `Future`.

await

Suspende a função até o `Future` terminar, liberando o *event loop* para outros eventos. Só pode ser usado dentro de funções `async`.

Erro comum

Esquecer o `await` faz o código seguir sem o resultado. O *lint* `unawaited_futures` ajuda a encontrar esses casos.

Erros, tempo limite e paralelismo

Tratando falhas

```dart
Future<void> carregar() async {
  try {
    final dados = await api.bancadas()
        .timeout(const Duration(seconds: 10));
    mostrar(dados);
  } on TimeoutException {
    mostrarErro('O servidor demorou demais');
  } on SocketException {
    mostrarErro('Sem conexão');
  } catch (e, pilha) {
    registrar(e, pilha);
  }
}
```

Em paralelo

```dart
final (temp, umid) = await (
  lerTemperatura(), lerUmidade()
).wait;             // os dois rodam ao mesmo tempo
```

Recurso	Uso
`.timeout(d)`	falha após um tempo
`Future.wait([..])`	espera vários
`(a, b).wait`	idem, com *records*
`Future.any`	o primeiro que terminar
`Future.delayed`	espera um tempo

Dica

Sequencial: tempo total = soma. Em paralelo: tempo total ≈ a mais lenta.

Cancelamento

`Future` não se cancela: use `Stream` ou o `CancelToken` do Dio (Módulo 11).

Streams

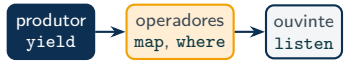
`Stream`: vários valores no tempo

Simulando um sensor

```dart
Stream<double> temperaturas() async* {
  var t = 24.0;
  final rnd = Random();
  while (true) {
    t += rnd.nextDouble() * 0.6 - 0.3;
    yield t;                          // emite um valor
    await Future.delayed(const Duration(seconds: 1));
  }
}

final inscricao = temperaturas()
    .map((t) => t.toStringAsFixed(1))
    .listen((texto) => print('$texto °C'));

// ao sair da tela:
await inscricao.cancel();
```



Future *versus* Stream

`Future`: **um** valor. `Stream`: **vários** valores ao longo do tempo (leituras de sensor, mensagens MQTT, mudanças no banco).

Atenção

Sempre cancele a inscrição (`cancel()`) quando não precisar mais dos dados; senão o sensor continua gastando bateria.

Operadores e StreamController

Pipeline reativo

```dart
final alertas = temperaturas()
    .map((t) => double.parse(t.toStringAsFixed(1)))
    .distinct()                          // ignora repetidos
    .where((t) => t > 30)
    .map((t) => 'Alerta: $t °C');

await for (final a in alertas.take(3)) {
  print(a);                              // consome 3 e para
}
```

StreamController

```dart
class Barramento {
  final _ctrl = StreamController<String>.broadcast();
  Stream<String> get eventos => _ctrl.stream;
  void publicar(String e) => _ctrl.add(e);
  void fechar() => _ctrl.close();
}
```

Operador	Efeito
`map`, `where`	transformar e filtrar
`distinct`	descarta repetidos seguidos
`take`, `first`	limita a quantidade
`asyncMap`	transforma com `Future`
`handleError`	trata erros
`timeout`	erro se ficar sem eventos

Simples *versus* broadcast

Uma `Stream` simples aceita **um** ouvinte; uma *broadcast* aceita vários.

Dica

O pacote `rxdart` acrescenta `debounceTime`, `combineLatest` e outros operadores.

Assíncrono na interface

FutureBuilder e StreamBuilder

Painel em tempo real

```
class PainelTemperatura extends StatelessWidget {
  const PainelTemperatura({super.key, required this.leituras});
  final Stream<double> leituras;

  @override
  Widget build(BuildContext context) => StreamBuilder<double>(
    stream: leituras,
    builder: (context, snap) {
      if (snap.hasError) return Text('Erro: ${snap.error}');
      if (!snap.hasData) {
        return const CircularProgressIndicator();
      }
      return Text('${snap.data!.toStringAsFixed(1)} °C',
          style: Theme.of(context).textTheme.displayMedium);
    },
  );
}
```

AsyncSnapshot

`snap.connectionState`, `hasData`, `data`, `hasError` e `error` permitem desenhar os estados carregando, sucesso e erro.

Future criado no `build`

`FutureBuilder(future: api.carregar())` refaz a requisição a cada reconstrução. Crie o `Future` no `initState` ou no *view model* (Módulo 08).

Temporizadores e `mounted`

Atualização periódica

```dart
class _PainelState extends State<Painel> {
  Timer? _timer;
  double? _temp;

  @override
  void initState() {
    super.initState();
    _timer = Timer.periodic(const Duration(seconds: 5),
        (_) => _atualizar());
  }

  Future<void> _atualizar() async {
    final t = await api.temperatura();
    if (!mounted) return;            // a tela ainda existe?
    setState(() => _temp = t);
  }

  @override
  void dispose() { _timer?.cancel(); super.dispose(); }
  // build(...)
}
```

Depois de um `await`

O usuário pode ter saído da tela. Verifique `mounted` antes de chamar `setState` ou usar o `context`.

Limpeza

Tudo que é criado no `initState` (timers, inscrições, controladores) é liberado no `dispose`.

Isolates

Isolates: CPU sem travar a interface

Cálculo pesado em outro isolate

```dart
// função pura, sem acesso à UI
List<double> fft(List<double> amostras) {
  // ... milhares de operações
  return espectro;
}

Future<void> analisar() async {
  final espectro = await Isolate.run(
      () => fft(amostras));     // roda em paralelo
  setState(() => _espectro = espectro);
}

// atalho do Flutter (também funciona na web)
final dados = await compute(decodificarJson, textoGrande);
```



Isolate

Unidade de execução com **memória própria**: não compartilha variáveis, troca mensagens. Por isso não há condições de corrida.

Quando usar

FFT de sinais, filtros, compressão de imagem, JSON muito grande (centenas de KB) e criptografia.

Fechamento

Resumo

Resumo

- O Dart roda o app num *event loop* de uma thread; nada pode bloqueá-lo por muito tempo.
- `Future` representa um valor futuro; `async`/`await` deixam o código sequencial e legível.
- Trate erros com `try`/`on`/`catch`, use `timeout` e paralelize com `Future.wait`.
- `Stream` emite vários valores; cancele as inscrições quando terminar.
- `FutureBuilder`/`StreamBuilder` desenham os estados; verifique `mounted` após `await`.
- Cálculo pesado vai para outro isolate com `Isolate.run` ou `compute`.

Glossário (1/2)

Termo	Significado
Thread	linha de execução dentro de um processo, com pilha própria
Isolate	unidade de execução do Dart com memória própria; comunica-se por mensagens
Laço de eventos	processa os eventos da fila, um de cada vez, na mesma *thread*
Microtarefa	tarefa curta executada antes do próximo evento da fila
Concorrência	tarefas em andamento no mesmo período, que se alternam
Paralelismo	tarefas executando no mesmo instante, em núcleos diferentes
`Future`	resultado que ainda vai existir: pendente, com valor ou com erro
`async/await`	escrevem código assíncrono com cara de sequencial
Jank	quadro perdido porque a *thread* da interface ficou ocupada

Glossário (2/2)

Termo	Significado
`Stream`	fluxo de vários valores ao longo do tempo
`StreamSubscription`	inscrição em um fluxo; deve ser cancelada ao terminar
Broadcast	fluxo quente que aceita vários ouvintes
`StreamController`	cria e alimenta um fluxo manualmente
Condição de corrida	resultado que depende da ordem imprevisível dos eventos
Contrapressão	estratégia para quando o produtor é mais rápido que o consumidor
`Isolate.run`	executa uma função em outro *isolate* e devolve o resultado

Questões de revisão

1. Se cada *isolate* tem uma única *thread*, como o app faz uma requisição de rede sem travar a tela?
2. Diferencie concorrência de paralelismo. Qual deles exige outro *isolate* no Dart?
3. Por que um laço de CPU pesado trava a interface mesmo dentro de uma função `async`?
4. O que acontece no laço de eventos quando a execução chega a um `await`?
5. Quem roda primeiro: microtarefas ou eventos? Por que isso importa?
6. Dê um exemplo de condição de corrida em Dart, mesmo com uma *thread* só, e como evitá-la.
7. Qual a diferença entre um `Stream` de inscrição única e um *broadcast*?
8. Por que *isolates* não precisam de travas para proteger seus dados?

Exercícios

Exercício 1 — Paralelismo

Escreva três funções que simulam leituras com atrasos de 1 s, 2 s e 3 s. Meça o tempo total em sequência e com `Future.wait`.

Exercício 2 — Stream

Crie uma `Stream<int>` que simula um ADC de 12 bits a 10 Hz e um pipeline que converte para volts, agrupa 10 amostras, calcula a média e emite alerta acima de 3,0 V.

Exercício 3 — Isolate

Gere 1 milhão de amostras e calcule estatísticas na thread principal com uma animação girando na tela. Repita com `Isolate.run` e compare.

LabMonitor — Etapa 6

Implemente `SimuladorSensor` com uma `Stream<Leitura>` por bancada e mostre a última leitura de cada bancada com `StreamBuilder`.

Para aprofundar

- Programação assíncrona: `https://dart.dev/libraries/async/async-await`
- Streams: `https://dart.dev/libraries/async/using-streams`
- Concorrência e isolates: `https://dart.dev/language/concurrency`
- Isolates no Flutter: `https://docs.flutter.dev/perf/isolates`
- Pacote rxdart: `https://pub.dev/packages/rxdart`

Respostas das questões de revisão

1. A requisição é I/O assíncrono: o Dart a entrega ao sistema operacional e segue atendendo eventos; quando a resposta chega, um evento completa o `Future`.
2. Concorrência: tarefas que se alternam no mesmo período (vários `Futures` no laço). Paralelismo: execução simultânea em núcleos diferentes, que no Dart exige outro *isolate*.
3. `async` não cria *thread*: o trecho entre dois `await` roda até o fim no laço de eventos. Um cálculo longo ocupa a *thread* e impede os quadros; use `Isolate.run`.
4. A função pausa e devolve o controle ao laço, que atende outros eventos; quando o `Future` completa, a continuação é agendada e a função segue do mesmo ponto.
5. A fila de microtarefas é esvaziada antes do próximo evento. Microtarefas demais atrasam toques e quadros, como um laço longo.
6. Duas buscas por texto: a resposta da primeira chega depois da segunda e sobrescreve o resultado mais novo. Guarde o identificador da última busca e ignore respostas antigas.
7. Inscrição única aceita um só ouvinte e começa a produzir quando alguém escuta; o *broadcast* aceita vários, e quem chega tarde perde os eventos anteriores.
8. Cada *isolate* tem memória própria e só troca cópias por mensagens: nenhum objeto mutável é acessado por dois *isolates* ao mesmo tempo.

Fim do Módulo 06

Próximo: Módulo 07 — Estado, navegação e animações: StatefulWidget, ciclo de vida, go_router e animações.